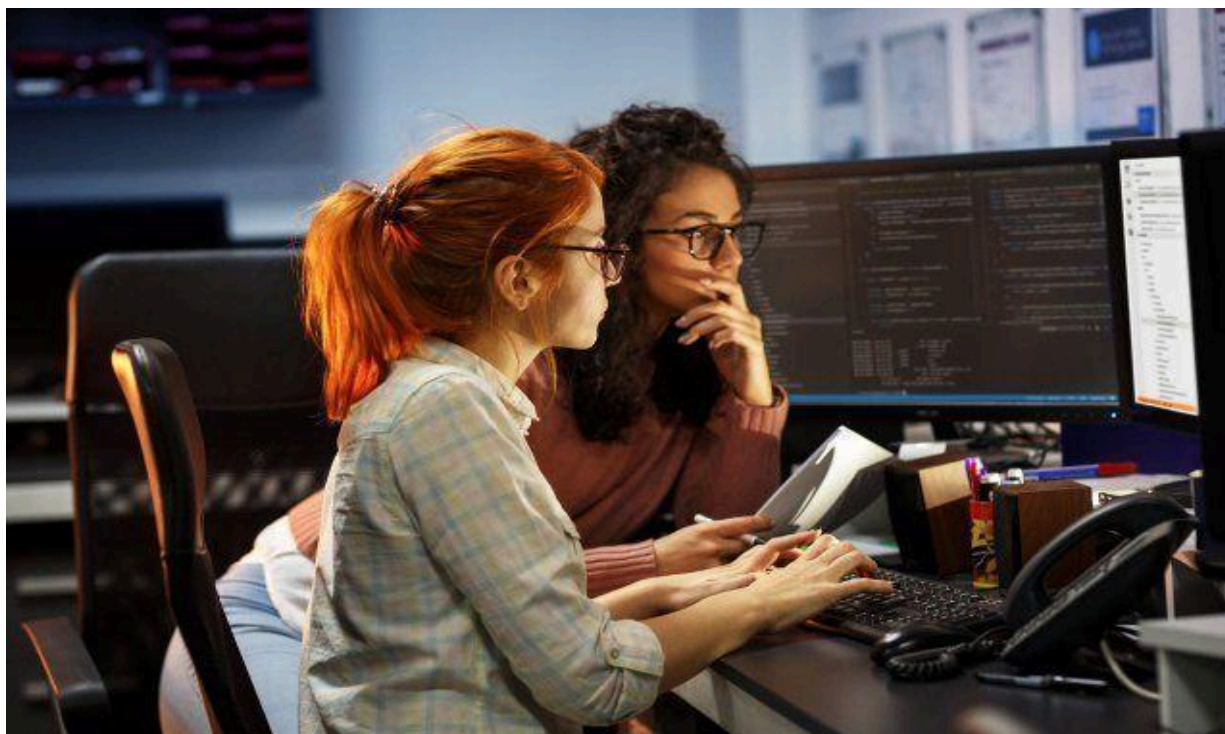




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 3

Tema: Programacion Orientada a Objetos en Desarrollo Moderno.

SEMANA 11

Autor:

Introducción:

Bienvenidos al material de lectura #3 de la Unidad #3 de la materia Programación Orientada a Objetos. En este documento exploramos más acerca de arquitectura en capas y arquitectura limpia, también veremos como usar H2 y hacer conexiones a BD relacionales y No relacionales . Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en los ejemplos, ventajas, limitaciones y ejercicios propuestos, ya que contribuirán significativamente a tu proceso de aprendizaje.

Semana 11

3.11 Principios de Arquitectura Limpia en Spring Boot

La **Arquitectura Limpia**, popularizada por Robert C. Martin ("Uncle Bob"), es un enfoque de diseño de software que promueve una estricta **separación de responsabilidades**. Su meta es crear sistemas independientes de frameworks, interfaces de usuario y bases de datos.

El principio clave es que el núcleo de la aplicación —la **lógica de negocio**— no debe depender de detalles externos. Esto hace que el sistema sea más fácil de probar, mantener y evolucionar.

Principios Clave:

1. **Independencia de Frameworks:** Tu lógica de negocio no debe saber que existe Spring Boot. Spring es una herramienta, no el centro del universo.
2. **Testable (Probable):** Las reglas de negocio, como "un producto no puede tener precio negativo", se pueden probar sin una base de datos o una API.
3. **Independencia de la UI:** La interfaz de usuario puede cambiar sin afectar cómo se calcula el inventario de un producto.
4. **Independencia de la Base de Datos:** Puedes migrar de H2 a MySQL sin modificar la clase **Producto**.

La Regla de la Dependencia

Este es el pilar de la Arquitectura Limpia. Se visualiza como círculos concéntricos donde las dependencias solo pueden apuntar hacia adentro.

- **Círculos Internos (Núcleo):** Contienen la lógica de negocio. Por ejemplo, la clase **Producto** con sus reglas de validación.
- **Círculos Externos (Detalles):** Contienen las implementaciones técnicas como Spring Boot, la base de datos PostgreSQL, etc.

Nada en un círculo interno puede conocer algo de un círculo externo. Tu entidad **Producto** no debería tener anotaciones de Spring.

Capas en la Arquitectura Limpia:

1. **Entidades (Entities):** Objetos de negocio puros. *Ejemplo: Una clase **Producto** con atributos como **nombre**, **precio** y **stock**.*

2. **Casos de Uso (Use Cases):** Orquestan el flujo de datos. *Ejemplo: Un caso de uso `CrearProducto` que valida los datos y lo guarda.*
3. **Adaptadores de Interfaz (Interface Adapters):** Convierten datos. Aquí viven los **Controladores**, **Presentadores** y **Repositorios**.
4. **Frameworks y Drivers:** La capa más externa. Herramientas como Spring Boot, H2, etc.

3.12 Separación en Capas: Controlador, Servicio, Repositorio

Esta es la implementación más común de la Arquitectura Limpia en Spring Boot, dividiendo la aplicación en tres capas lógicas.

1. Capa de Controlador (@RestController)

- **Responsabilidad:** Manejar las peticiones HTTP (GET, POST, etc.), validar datos de entrada (DTOs) y devolver una respuesta JSON. Es el portero de tu API.
- **Características:**
 - No tiene lógica de negocio.
 - Llama a la capa de servicio para hacer el trabajo real.
 - Mapea URLs como `/api/productos` a métodos específicos.

Ejemplo de Controlador:

```
import
org.springframework.beans.factory.annotation.Autowired;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController

@RequestMapping("/api/productos")

public class ProductoController {
```

```
@Autowired

private ProductoService productoService;

@PostMapping

    public      ResponseEntity<Producto>
crearProducto(@RequestBody ProductoDT0 productoDT0) {

        Producto      nuevoProducto      =
productoService.crearProducto(productoDT0);

                                                                    return
ResponseEntity.status(201).body(nuevoProducto);

    }

@GetMapping("/{id}")

    public      ResponseEntity<Producto>
obtenerProductoPorId(@PathVariable Long id) {

        return productoService.obtenerPorId(id)

            .map(ResponseEntity::ok) // Si lo encuentra,
devuelve 200 OK

            .orElse(ResponseEntity.notFound().build());
// Si no, 404 Not Found

    }

@GetMapping

    public List<Producto> obtenerTodosLosProductos() {
```

```
        return productoService.obtenerTodos();  
    }  
}
```

2. Capa de Servicio (@Service)

- **Responsabilidad:** Contiene la lógica de negocio principal. Orquesta las operaciones, aplica reglas y decide a qué repositorios llamar. Es el cerebro de la aplicación.
- **Características:**
 - Puede combinar llamadas a varios repositorios.
 - Gestiona las transacciones (@Transactional).

Ejemplo de Servicio:

```
import  
org.springframework.beans.factory.annotation.Autowired;  
  
import org.springframework.stereotype.Service;  
  
import  
org.springframework.transaction.annotation.Transactional;  
  
import java.util.List;  
  
import java.util.Optional;  
  
  
@Service  
  
public class ProductoService {  
  
    @Autowired  
  
    private ProductoRepository productoRepository;  
  
    @Transactional  
  
    public Producto crearProducto(ProductoDTO productoDTO) {
```

```
// Lógica de negocio: no permitir productos con  
precio cero o negativo
```

```
    if (productoDTO.getPrecio() <= 0) {  
  
        throw new IllegalArgumentException("El precio  
debe ser mayor a cero.");  
    }
```

```
// Lógica de negocio: no permitir productos  
duplicados por SKU
```

```
    if  
(productoRepository.findBySku(productoDTO.getSku()).isPresent  
( )) {  
  
        throw new IllegalStateException("El SKU ya  
existe.");  
    }
```

```
    Producto producto = new Producto();  
    producto.setNombre(productoDTO.getNombre());  
    producto.setPrecio(productoDTO.getPrecio());  
    producto.setSku(productoDTO.getSku());
```

```
    return productoRepository.save(producto);  
}
```

```
@Transactional(readonly = true)
```

```
public List<Producto> obtenerTodos() {
```

```
        return productoRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Optional<Producto> obtenerPorId(Long id) {
        return productoRepository.findById(id);
    }
}
```

3. Capa de Repositorio (@Repository)

- **Responsabilidad:** Abstraer el acceso y la manipulación de datos en la base de datos.
- **Características:**
 - Define métodos para operaciones CRUD (Crear, Leer, Actualizar, Borrar).
 - Spring Data JPA lo implementa por ti.

Ejemplo de Repositorio:

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
```

@Repository

```
public interface ProductoRepository extends
    JpaRepository<Producto, Long> {
```

// Spring Data JPA crea esta consulta automáticamente:

```
// SELECT * FROM producto WHERE sku = ?
```

```
Optional<Producto> findBySku(String sku);  
  
}
```

3.13 Streams y Lambdas

Introducidos en Java 8, los **Streams** y **lambdas** permiten un estilo de programación **funcional y declarativo**, haciendo el código para manejar colecciones más limpio y legible.

¿Qué es un Stream?

Un **Stream** es una **secuencia de elementos** que se pueden procesar de forma **funcional y declarativa**.

No modifica los datos originales, sino que **crea un flujo de datos** sobre los cuales se aplican operaciones.

En lugar de decir “*cómo hacerlo*” (con **for**, **if**, etc.), tú dices “*qué quiero obtener*”.

Cómo funciona internamente

Podemos pensar en el **Stream** como un **pipeline (tubería)** de tres fases:

FUENTE → **OPERACIONES INTERMEDIAS** → **OPERACIÓN TERMINAL**

1. Fuente de datos

Puede venir de:

- Una colección (**List**, **Set**)

- Un arreglo
- Un archivo
- Una función generadora

```
List<String> nombres = List.of("Ana", "Juan", "Pedro",  
"Sofía");
```

```
Stream<String> flujo = nombres.stream();
```

2. Operaciones intermedias

Transforman o filtran los datos, **sin ejecutar todavía nada**.

Son **perezosas (lazy)**: no hacen nada hasta que haya una operación terminal.

Ejemplos:

- `filter()` → filtra elementos
- `map()` → transforma elementos
- `sorted()` → los ordena
- `distinct()` → elimina duplicados

```
flujo = flujo  
    .filter(n -> n.length() > 3)  
    .map(String::toUpperCase)  
    .sorted();
```

En este punto, todavía **no se ejecuta nada**. Solo se define el *pipeline*.

3. Operación terminal

Es la que **activa el procesamiento** del Stream.

Ejemplos:

- `toList()`, `collect()` → convierten a lista
- `count()` → devuelve número de elementos
- `forEach()` → ejecuta una acción
- `findFirst()`, `anyMatch()`, etc.

```
List<String> resultado = flujo.toList();
```

En este momento el **Stream se consume** (ya no se puede reutilizar).

⚙️ Tipos principales de operaciones terminales			
Categoría	Método	Qué hace	Tipo de resultado
Recolección	<code>collect()</code> / <code>toList()</code>	Recolecta los elementos en una colección	<code>List</code> , <code>Set</code> , <code>Map</code> , etc.
Iteración	<code>forEach()</code> / <code>forEachOrdered()</code>	Ejecuta una acción sobre cada elemento	<code>void</code>
Conteo	<code>count()</code>	Cuenta los elementos del Stream	<code>long</code>
Búsqueda / coincidencia	<code>findFirst()</code> , <code>findAny()</code> , <code>anyMatch()</code> , <code>allMatch()</code> , <code>noneMatch()</code>	Busca o verifica condiciones	<code>Optional</code> o <code>boolean</code>
Reducción	<code>reduce()</code>	Combina todos los elementos en un único valor	<code>Optional<T></code>
Resumen numérico	<code>sum()</code> , <code>average()</code> , <code>min()</code> , <code>max()</code> (en <code>IntStream</code> , <code>DoubleStream</code> ...)	Calcula estadísticas	<code>int</code> , <code>double</code> , <code>Optional</code>

Ejemplo completo para entender streams

```
List<String> nombres = List.of("Ana", "Juan", "Pedro",  
"Sofía");
```

```
List<String> resultado = nombres.stream()    // Fuente  
    .filter(n -> n.length() > 3)    // Intermedia 1  
    .map(String::toUpperCase)    // Intermedia 2  
    .sorted()    // Intermedia 3  
    .toList();    // Terminal
```

```
System.out.println(resultado); // [JUAN, PEDRO, SOFÍA]
```

Expresiones Lambda

Una lambda es una función anónima y concisa.

Sintaxis: (parametros) -> { cuerpo }

- **Ejemplo con lambda:** `producto -> producto.getPrecio() > 100.0`
- **Equivalente sin lambda:**

```
new Predicate<Producto>() {  
    @Override  
    public boolean test(Producto producto) {  
        return producto.getPrecio() > 100.0;  
    }  
}
```

En Java, las *lambdas* son expresiones que representan funciones anónimas (sin nombre), y se usan normalmente donde se espera una interfaz funcional —es decir, una interfaz con un solo método abstracto.

Veamos los tipos principales de lambdas según el tipo de interfaz funcional que implementan 🙋

1. Según su forma sintáctica

♦ a) Sin parámetros

```
() -> System.out.println("Hola mundo");
```

👉 Se usa cuando el método no recibe argumentos.
Ejemplo:

```
Runnable tarea = () -> System.out.println("Ejecutando tarea...");
```

♦ b) Con un parámetro

```
x -> x * x
```

No se necesitan paréntesis si hay un solo parámetro.
Ejemplo:

```
Consumer<String> imprimir = x -> System.out.println(x);
```

♦ c) Con varios parámetros

```
(a, b) -> a + b
```

Ejemplo:

```
BinaryOperator<Integer> sumar = (a, b) -> a + b;
```

♦ d) Con cuerpo de varias líneas

Si la expresión tiene más de una instrucción, se usan llaves {}:

```
(x, y) -> {
    int suma = x + y;
    return suma * 2;
}
```

2. Según la interfaz funcional estándar (java.util.function)

Java trae muchas interfaces funcionales en el paquete `java.util.function`.

Aquí están las más usadas 📌

Tipo	Firma del método	Ejemplo	Descripción
<code>Predicate<T></code>	<code>boolean test(T t)</code>	<code>x -> x > 10</code>	Devuelve <code>true</code> o <code>false</code> (para filtrado)
<code>Function<T, R></code>	<code>R apply(T t)</code>	<code>x -> x * 2</code>	Transforma un valor en otro
<code>Consumer<T></code>	<code>void accept(T t)</code>	<code>x -></code> <code>System.out.println(x)</code>	Consume el valor sin devolver nada
<code>Supplier<T></code>	<code>T get()</code>	<code>() -> Math.random()</code>	No recibe nada y devuelve un valor
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	<code>x -> x * x</code>	Igual tipo de entrada y salida
<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>	<code>(a, b) -> a + b</code>	Opera con dos valores del mismo tipo
<code>BiFunction<T,U,R></code>	<code>R apply(T t, U u)</code>	<code>(x, y) -> x + " " + y</code>	Dos entradas de tipos distintos, un resultado

Resumen:

Tipos de lambdas

Sin parámetros

```
() -> System.out.println("Hola mundo")
```

Runnable

Con un parámetro

```
x -> x * x
```

Function<T, R>

Con varios parámetros

```
(a, b) -> a + b
```

BinaryOperator<T>

Con cuerpo de varias líneas

```
(x, y) -> { int suma = x + y;  
            return suma * 2;
```

BinaryOperator<T>

Ejemplo Práctico: De una lista de productos, obtener los nombres de los que pertenecen a la categoría "Electrónica" y cuestan menos de 500, ordenados por precio.

```
import java.util.Arrays;  
  
import java.util.List;  
  
import java.util.stream.Collectors;  
  
import java.util.Comparator;  
  
  
public class EjemploStreams {  
    public static void main(String[] args) {
```

```
List<Producto> productos = //... una lista de
productos
```

```
List<String> nombresDeOfertas = productos.stream() //
1. Convertir la lista a Stream
```

```
.filter(p ->
"Electrónica".equals(p.getCategoria())) // 2. Filtrar por
categoría
```

```
.filter(p -> p.getPrecio() < 500.0) // 3. Filtrar
por precio
```

```
.sorted(Comparator.comparing(Producto::getPrecio)) // 4.
Ordenar por precio ascendente
```

```
.map(Producto::getNombre) // 5. Mapear cada
objeto Producto a su nombre (String)
```

```
.collect(Collectors.toList()); // 6. Recolectar
los resultados en una nueva lista
```

```
nombresDeOfertas.forEach(System.out::println);
}
}
```

1. productos

productos es la **fuentes de datos**:

una lista de objetos, probablemente algo como esto:

```
List<Producto> productos = List.of(
    new Producto("Televisor", "Electrónica", 500),
```

```
new Producto("Zapatos", "Moda", 50),  
new Producto("Laptop", "Electrónica", 1000)  
);
```

Es decir, cada elemento de la lista es un objeto de tipo **Producto**.

2. **.stream()**

Convierte la lista en un **flujo (Stream)** para poder usar operaciones como **filter()**, **map()**, **sorted()**, etc.

```
productos.stream()
```

👉 A partir de este punto, el Stream trabajará **elemento por elemento**.

3. **filter(p** -> **"Electrónica".equals(p.getCategoria()))**

Aquí aparece la **lambda** y su parámetro **p**.

¿Qué es **p**?

p es el **nombre del parámetro** que representa **cada elemento del Stream**, es decir, **cada objeto Producto** dentro de la lista original.

Podrías leerlo así:

"De todos los productos, **toma aquellos** donde la categoría sea 'Electrónica'".

En forma expandida:


```
.filter(p -> "Electrónica".equals(p.getCategoria()))
```

es equivalente a:

```
.filter((Producto p) -> {  
    return "Electrónica".equals(p.getCategoria());  
})
```

Y también equivalente a un bucle tradicional:

```
for (Producto p : productos) {  
    if ("Electrónica".equals(p.getCategoria())) {  
        // pasa el filtro  
    }  
}
```

4. nombresDeOfertas

Es la **lista final resultante** después de aplicar el Stream completo.

En tu fragmento, solo se muestra la parte del filtro, pero el flujo completo probablemente sigue con algo como:

```
List<String> nombresDeOfertas = productos.stream()  
    .filter(p -> "Electrónica".equals(p.getCategoria())) //  
    ♦ deja solo electrónicos  
    .map(p -> p.getNombre()) //  
    ♦ toma solo el nombre
```

```
.toList();
```

- ◆ convierte a lista

Resultado:

```
["Televisor", "Laptop"]
```

Elemento	Qué representa	Tipo
<code>productos</code>	Lista original de objetos	<code>List<Producto></code>
<code>p</code>	Cada producto individual dentro del Stream	<code>Producto</code>
<code>p -> condición</code>	Expresión lambda que evalúa cada producto	<code>Predicate<Producto></code>
<code>nombresDeOfertas</code>	Lista final filtrada o transformada	<code>List<String></code>

3.14 Uso de Spring Data JPA

Spring Data JPA es una maravilla que simplifica la capa de persistencia, eliminando casi todo el código repetitivo.

Características Principales:

- **Repositorios Declarativos:** Solo defines una interfaz que hereda de `JpaRepository` y Spring crea la implementación.
- **Consultas Derivadas:** Spring crea consultas SQL a partir del nombre de los métodos. `findByCategoriaOrderByNombreAsc` se convierte en `SELECT * FROM producto WHERE categoria = ? ORDER BY nombre ASC`.
- **Paginación y Ordenamiento:** Soporte nativo para manejar grandes resultados.
- **Consultas Personalizadas:** Permite usar JPQL (`@Query`) para consultas complejas.

Ejemplo de Interfaz de Repositorio Avanzada:

Java

```
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import org.springframework.stereotype.Repository;

import java.util.List;

@Repository

public interface ProductoRepository extends
JpaRepository<Producto, Long> {

    // Consulta derivada por nombre de método

    List<Producto> findByCategoria(String categoria);

    // Consulta derivada más compleja

                                List<Producto>
    findByPrecioLessThanAndStockGreaterThan(double precioMax, int
stockMin);

    // Consulta personalizada con JPQL para buscar por nombre
(case-insensitive)

    @Query("SELECT p FROM Producto p WHERE lower(p.nombre)
LIKE lower(concat('%', :texto, '%'))")

    List<Producto> buscarPorNombre(String texto);
}
```

3.15 Conexión con Bases de Datos

Spring Boot hace que configurar la conexión a una base de datos sea trivial.

1. Conexión con H2 (en memoria)

Ideal para desarrollo y pruebas. Se crea y destruye con la aplicación.

1. Dependencia Maven (**pom.xml**):

```
<dependency>

    <groupId>com.h2database</groupId>

    <artifactId>h2</artifactId>

    <scope>runtime</scope>

</dependency>
```

2. Configuración (**application.properties**):

URL de conexión para H2 en memoria

```
spring.datasource.url=jdbc:h2:mem:testdb
```

```
spring.datasource.username=sa
```

```
spring.datasource.password=password
```

```
spring.datasource.driverClassName=org.h2.Driver
```

Habilitar la consola web de H2

```
spring.h2.console.enabled=true
```

Política de creación de tablas de Hibernate (ideal para desarrollo)

```
spring.jpa.hibernate.ddl-auto=update
```

Accede a la consola H2 en <http://localhost:8080/h2-console>.

2. Conexión con Base de Datos Relacional (JDBC)

Para conectar a una base de datos como **PostgreSQL**, solo cambia el driver y la configuración.

1. Dependencia Maven (**pom.xml**):

```
<dependency>  
    <groupId>org.postgresql</groupId>  
    <artifactId>postgresql</artifactId>  
    <scope>runtime</scope>  
</dependency>
```

2. Configuración (**application.properties**): Properties

URL de conexión JDBC para PostgreSQL

```
spring.datasource.url=jdbc:postgresql://localhost:5432/tienda_db
```

```
spring.datasource.username=postgres
```

```
spring.datasource.password=tu_contraseña
```

En producción, se recomienda usar 'validate' o 'none'

```
spring.jpa.hibernate.ddl-auto=update
```

3. Conexión con MongoDB (NoSQL)

Para bases de datos NoSQL, se usa **Spring Data MongoDB**.

1. Dependencia Maven (**pom.xml**):

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-mongodb</artifactId>  
</dependency>
```

2. Configuración (**application.properties**):

URI de conexión de MongoDB

```
spring.data.mongodb.uri=mongodb://localhost:27017/tienda_mongo_db
```

En MongoDB, las entidades se anotan con `@Document` y los repositorios extienden `MongoRepository`.

3.16 Anotaciones: `@Entity`, `@Id`, `@GeneratedValue`, `@Repository`

Estas son las anotaciones clave para la persistencia con JPA en Spring.

`@Entity`

- **Paquete:** `jakarta.persistence.Entity`
- **Uso:** Le dice a JPA que una clase es una **entidad** y que debe ser mapeada a una tabla en la base de datos.

@Id

- **Paquete:** `jakarta.persistence.Id`
- **Uso:** Marca el campo que funcionará como **clave primaria** de la tabla.

@GeneratedValue

- **Paquete:** `jakarta.persistence.GeneratedValue`
- **Uso:** Especifica cómo se generará el valor de la clave primaria.
- **Estrategias:** `IDENTITY` (autoincremento de la BD), `SEQUENCE` (secuencia de la BD), `AUTO` (JPA decide).

@Repository

- **Paquete:** `org.springframework.stereotype.Repository`
- **Uso:** Marca una interfaz o clase como un componente de la capa de acceso a datos.
- **Beneficios:**
 1. **Inyección de Dependencias:** Permite que Spring lo gestione e inyecte en otras clases (ej. en un `@Service`).
 2. **Traducción de Excepciones:** Convierte excepciones de bajo nivel (como `SQLException`) en excepciones de Spring más manejables.

Ejemplo Completo de una Entidad **Producto**:

```
import jakarta.persistence.Entity;  
  
import jakarta.persistence.GeneratedValue;  
  
import jakarta.persistence.GenerationType;  
  
import jakarta.persistence.Id;  
  
import jakarta.persistence.Table;  
  
import jakarta.persistence.Column;
```

```
@Entity // 1. Esta clase es una entidad JPA.
```

```
@Table(name = "productos") // 2. Mapea esta entidad a la  
tabla "productos".
```

```
public class Producto {
```

```
    @Id // 3. Este es el campo de la clave primaria.
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY) // 4.  
    La BD generará el ID automáticamente.
```

```
    private Long id;
```

```
    @Column(nullable = false, unique = true)
```

```
    private String sku; // Stock Keeping Unit
```

```
    @Column(nullable = false, length = 100)
```

```
    private String nombre;
```

```
    private String categoria;
```

```
    private double precio;
```

```
    private int stock;
```

```
    // Constructor vacío (requerido por JPA)
```

```
    public Producto() {
```



```
}
```

```
// Getters y Setters...
```

```
public Long getId() { return id; }
```

```
public void setId(Long id) { this.id = id; }
```

```
public String getSku() { return sku; }
```

```
public void setSku(String sku) { this.sku = sku; }
```

```
public String getNombre() { return nombre; }
```

```
    public void setNombre(String nombre) { this.nombre =  
nombre; }
```

```
public String getCategoria() { return categoria; }
```

```
    public void setCategoria(String categoria) {  
this.categoria = categoria; }
```

```
public double getPrecio() { return precio; }
```

```
    public void setPrecio(double precio) { this.precio =  
precio; }
```

```
public int getStock() { return stock; }
```

```
public void setStock(int stock) { this.stock = stock; }
```

```
}
```